Musubi Japan

Laboratorio: arquitectura, subarquitectura, SOLID y patrones

Ingeniería de Software II (2016702)

Daniel Gracia

Octubre de 2026

`Lab_Patrones_GrupoXX.pdf`

1. Fase 1. Dominio y problema

1.1. Sector profesional

El sector es la educación de un segundo idioma. Dentro de él, el japonés se distingue del español porque la escritura, el léxico y la gramática no se recorren con un solo alfabeto. El hiragana y el katakana son silabarios: permiten leer una frase en voz alta sin conocer el significado de las palabras. El kanji y el vocabulario cargan ese significado, y para leer y escribir un texto común hace falta conocer más de 2000 caracteres. La gramática, además, sigue una lógica distinta de la de las lenguas romances, y eso pesa cuando quien estudia tiene el español como primera lengua. Esas partes se enseñan y se practican a menudo por separado, aunque en un texto aparecen juntas. El avance no se mide solo con palabras aisladas: reconocer una pieza es distinto de usarla bien en un contexto.

1.2. Actores

- **Aprendiz de japonés.** Quien estudia el idioma en el celular, sea novato o ya tenga conocimientos previos, y busca una herramienta de reforzamiento.
- **Proveedor de IA.** Sistema externo de la capa de acompañamiento. Sostiene a la mascota cuando hay red y el aprendiz tiene acceso a esa capa.
- **Red de anuncios y pasarela de pago.** Sistemas externos que financian la capa de IA, por anuncios o por una opción de pago. El estudio sin mascota no los usa.

1.3. Problema

El refuerzo del japonés está repartido en varias aplicaciones. Unas trabajan el kanji, otras el vocabulario y la gramática, otras los textos y las lecturas. Las que reúnen varios de esos componentes tienen un costo alto. Quien estudia salta de una herramienta a otra, y el kanji, la palabra, la gramática y el escrito quedan sueltos.

Musubi Japan reúne esos contenidos en un solo lugar y los entrega con la aplicación. El recorrido lo decide el aprendiz: sigue el camino establecido por la aplicación o incorpora, poco a poco, las frases de lo que va aprendiendo, en el orden que le sirve. Cada frase entra como material y la aplicación la desglosa en vocabulario, reglas gramaticales y kanji. Cada pieza guarda dos grados de avance: el reconocimiento, cuando la persona la identifica, y la interiorización, cuando ya la usa bien.

El ejercicio mueve solo las piezas que pone en juego. Dibujar el kanji de 食べる (食, en たべる, con el sentido general de comida) actualiza ese kanji. Producir りんごを食べます (como una manzana) actualiza las palabras りんご (manzana) y 食べる (comer), la partícula を (la acción recae sobre el objeto directo), el verbo en presente (食べます, forma cordial) y el kanji de 食べる.

El estudio cabe entero en el teléfono, sin red y sin mascota. En modo offline la interiorización es autoevaluativa y no hay anuncios. La mascota vive en otra capa. Cuando está disponible, acompaña el aprendizaje con contexto de uso cotidiano: cuándo conviene una frase y cuándo otra, o cuando conviene usar una palabra en un contexto diferente.

1.4. Requisitos funcionales

RF1. Como aprendiz novato, quiero seguir una ruta de aprendizaje sugerida para adentrarme en el aprendizaje del japonés.

RF2. Como aprendiz, quiero incorporar en mi orden las frases que voy aprendiendo para no olvidarlas y poder usarlas más adelante.

RF3. Como aprendiz, quiero que en cada pieza (vocabulario, kanji o gramática) se registre qué tanto la identifico y qué tanto soy capaz de usarla, para ver cómo voy con cada una.

RF4. Como aprendiz, quiero practicar una pieza aislada (vocabulario, kanji o gramática) para entender su significado y en qué otras frases puede usarse.

RF5. Como aprendiz, quiero que, al practicar una frase, el progreso de sus piezas (vocabulario, kanji y gramática) avance para dedicar menos tiempo a cada una.

RF6. Como aprendiz, quiero ver el progreso de mi avance general para saber qué tanto he avanzado en mi aprendizaje.

RF7. Como aprendiz, quiero acceder a lecturas en japonés para practicar lo que he aprendido y fortalecer mi aprendizaje del idioma.

RF8. Como aprendiz, quiero usar la aplicación sin conexión para no depender de los anuncios ni del uso obligatorio de la mascota.

RF9. Como aprendiz, quiero que la mascota me dé ejercicios nuevos e información de uso correcto para fortalecer mi uso del japonés.

RF10. Como sistema, quiero ofrecer la mascota mediante anuncios o pago para sostener su servicio.

1.5. Atributos de calidad

Disponibilidad. El aprendiz solicita la ruta, la práctica, las lecturas o el avance sin conexión; el sistema los entrega en el dispositivo en el 100 % de las solicitudes.

Rendimiento. El aprendiz termina de practicar una frase; el sistema muestra el progreso de sus piezas y el avance general en menos de un segundo.

Mantenibilidad. Los componentes de la aplicación permanecen desacoplados; cada componente y cada conexión entre ellos se prueba sin ejecutar el resto del sistema.

Seguridad. Si el aprendiz estudia sin conexión, el sistema conserva sus frases y su avance solo en el teléfono. Si usa la mascota, el sistema envía al proveedor únicamente la solicitud, sin información personal.

2. Fase 2. Arquitectura macro

Musubi Japan usa un monolito modular en el teléfono y un solo servicio para la mascota, desplegado en un servidor. La práctica, las lecturas, el detalle, el avance y el almacén local viven en la aplicación, junto con los grupos de frases. La ruta sugerida es el grupo que ofrece el sistema, ordenado por temas. El aprendiz puede seguirlo, crear sus propios grupos con las frases que considera relacionadas, o pasar de uno al otro. Al guardar una frase, una función del grupo la desglosa en hiragana, katakana, kanji, vocabulario y gramática, y asocia las piezas de ese grupo. Si una pieza ya trae avance de otro grupo, ese progreso sirve para enfocarse en lo que falta. El avance muestra el progreso general, luego cada tipo de pieza y, dentro, cómo va cada una. Desde una lectura o desde el avance, un elemento abre el detalle de su pieza: ahí se entiende y, si el aprendiz quiere, se incorpora a un grupo. El módulo de la mascota queda aparte del estudio. Con la respuesta del proveedor incorpora frases nuevas en un grupo, propone sesiones en la práctica y genera lecturas. Con red abre los anuncios o el pago y envía solo la solicitud al servicio de la mascota, que la reenvía al proveedor de IA. Ese servicio no guarda datos del aprendiz. El almacén local, cuando hay red, deja una copia de seguridad en el Google Drive del aprendiz. No hay gateway ni broker.

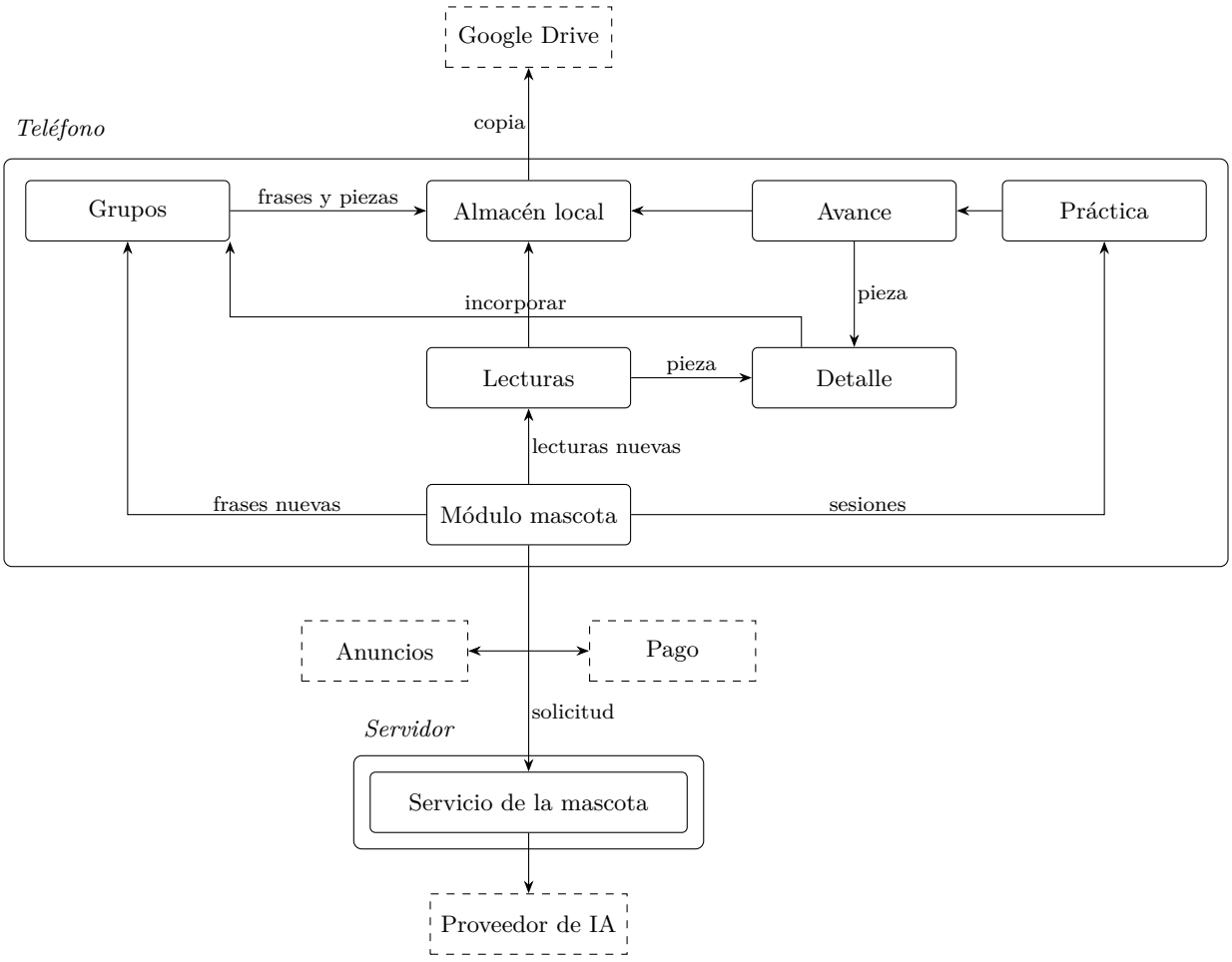


Figura 1. Arquitectura macro de Musubi Japan. Trazo continuo: componentes de Musubi Japan. Trazo punteado: sistemas externos.

2.1. Justificación frente a microservicios

Partir la ruta, la práctica, las lecturas y el avance en microservicios no cumple los escenarios de la fase 1.

Disponibilidad. Esas funciones responden sin red en el 100 % de las solicitudes. En servicios separados dejan de estar en el teléfono.

Rendimiento. El progreso de una frase se muestra en menos de un segundo, sin esperar a la red. Un servicio remoto no cumple esa medida.

Seguridad. Sin conexión, las frases y el avance se quedan en el teléfono. La única copia que sale, cuando hay red, es la del Google Drive del aprendiz. El servidor de la mascota no la guarda.

Mantenibilidad. Cada componente y cada conexión se prueban sin ejecutar el resto del sistema. Esa separación es entre módulos de la misma aplicación.

El servicio de la mascota sí se despliega aparte, en un servidor: la clave del proveedor no va en la aplicación, y los anuncios o el pago sostienen ese servicio. Sigue siendo un solo servicio.

3. Fase 3. Componente y subarquitectura

3.1. Componente elegido

Práctica es el componente que se diseña porque ahí el aprendiz repasa lo que ya vio y, cuando hay conexión, construye con más libertad. Los grupos, por su parte, se limitan a ordenar frases y a desglosarlas; Avance guarda los puntajes y los muestra, mientras que Lecturas y Detalle enseñan la pieza. La mascota tampoco reemplaza esa regla, pues solo llama la herramienta de Práctica y ofrece la sesión.

Repasar un kanji o un vocabulario es un ejercicio acotado: el aprendiz lo reconoce, lo identifica y lo escribe, a mano o con el teclado, y por eso ese repaso mueve solo esa pieza. Repasar gramática, en cambio, consiste en construir frases. Sin conexión, esa evaluación es autoevaluativa: el aprendiz juzga si la frase le quedó bien. Con conexión, el agente puede evaluar construcciones que el aprendiz no había hecho.

El modo libre extiende esa construcción. En él, el aprendiz arma frases o párrafos. Sin conexión, la evaluación también es autoevaluativa; con conexión, el agente los evalúa y los corrige. Así dispone de herramientas para poblar sus grupos con más frases, si así lo desea. La interiorización, entonces, queda solo en las piezas que usó bien en ese contexto, mientras que las demás reciben reconocimiento cuando la forma es correcta, pero no encaja en el contexto pedido. Además, un hiragana o un katakana ya interiorizado no vuelve a ese foco, de modo que el repaso y la corrección se concentran en el kanji, el vocabulario y la gramática que todavía lo necesitan.

3.2. MVVM

MVVM reparte Práctica en tres partes, y cada una cambia por su propia razón. La vista es el conjunto de pantallas de la práctica: cada una muestra su ejercicio y recoge lo que el aprendiz hace, sin decidir si eso sube el avance. El ViewModel dirige la sesión: mira qué quiso estudiar, le dice a la pantalla qué ejercicio mostrar y, al final, anota el resultado. El modelo reúne lo que se está practicando: la sesión, la pieza, la frase y la regla que dice qué puntaje sube. No sabe cómo se ve la pantalla ni quién abrió la práctica. La pantalla le entrega la respuesta a la sesión, la sesión le pregunta a la regla qué puntajes suben y los guarda en el avance.

Vista. `PantallaDePractica` es la interfaz que comparten las vistas: cada una muestra su ejercicio y recoge la respuesta, y todas la entregan a la misma sesión. `IIdentificacion`, `IRecuperacion` e `IProduccion` la extienden. En kana, kanji y vocabulario, la de identificar presenta un conjunto de distractores o un conjunto de traducciones, y la de interiorizar recoge la escritura, a mano o en el celular. En gramática, una pide ubicar partículas o auxiliares, u ordenar la frase, y otra recoge la frase producida con ese patrón. La producción libre recoge el texto y, sin conexión, también el juicio del aprendiz; no es una `PantallaDePractica`, porque no parte de una sola pieza. Ninguna calcula los puntajes.

ViewModel. `SesionDePractica` abre la sesión que el aprendiz configuró y recorre su plan, ítem por ítem. Para cada uno pide al catálogo de ejercicios la vista que corresponde al tipo de pieza y al grado; si es gramática, toma de la selección una frase que la use y trabaja sobre una copia. Lee en el almacén lo que esa selección necesita. Sin conexión, la frase de gramática y la producción libre quedan en autoevaluación. Con conexión, en la producción libre pide al agente que evalúe el texto; el contraste con el avance del aprendiz se hace en el teléfono, porque al proveedor solo sale la solicitud. Si la frase es nueva, presenta la conversación sobre cómo la conoce y si se incorpora al grupo. Luego pide el resultado al modelo y lo registra en Avance.

Modelo. Reúne `Sesion`, `Pieza`, `Progreso`, `Frase` y `ReglaDeEfecto`. La sesión y la frase dicen qué se está practicando, la pieza dice qué se estudia, el progreso cuánto lo domina el aprendiz y la regla decide qué sube.

- `Sesion.` Es la práctica en curso, distinta del ViewModel que la dirige. Guarda lo que el aprendiz eligió estudiar —un grupo, varios grupos o piezas sueltas— y cómo quiere hacerlo: qué tipos de pieza entran; si practica identificar, usar o ambos; un umbral de avance que deja solo las piezas por debajo de cierto valor o desde él; el orden —como en la selección, identificar antes de usar, primero las más bajas o las más altas, o al azar—, y, si quiere, un límite. Con eso arma el plan: una lista de ítems, cada uno una pieza en un grado. No calcula el puntaje.
- `Pieza.` Es una unidad de estudio y solo guarda contenido, el mismo para todos los aprendices. Tiene cuatro formas: `Kana`, hiragana o katakana; `Kanji`; `Vocabulario`, con los kanji que trae; y `Gramatica`. La gramática no es una palabra suelta sino una estructura: un patrón de partes fijas, huecos y partes opcionales, como [causa]。だから[consecuencia], donde la causa puede quedar en la conversación. Además lleva su registro —cordial, informal o indiferente—, de modo que だから y ですから son dos piezas. Cada pieza tiene un identificador estable. Una actualización de los contenidos reemplaza la pieza sin borrar el avance que el aprendiz lleva en ella.
- `Progreso.` Es lo que el aprendiz lleva en una pieza: por separado, qué tanto la identifica y qué tanto es capaz de usarla. Es un dato del aprendiz, no del contenido, y no sale del teléfono. Avance lo guarda por el identificador de la pieza. No se modifica: subir un puntaje produce un progreso nuevo. Los dos grados no son etapas: el ejercicio que el aprendiz elige decide cuál sube, y nada obliga a identificar antes de usar. Usar bien una pieza, eso sí, también cuenta como identificarla, para que quien prefiere producir no aparezca sin reconocimiento.
- `Frase.` Es el enunciado que se construye o se produce. Reúne las piezas que pone en juego y no tiene avance propio: practicarla mueve el de sus piezas. Si es nueva, no queda conocida hasta que el aprendiz decide incorporarla a un grupo, y solo `incorporarGrupo()` la marca como incorporada.

- `ReglaDeEfecto`. Recibe la respuesta de la sesión y devuelve qué puntaje sube en cada pieza, sin guardarlo. Elige la política por el tipo de pieza del ítem, y el ítem ya trae el grado. Identificar sube lo que el aprendiz reconoce; recuperar de memoria o producir sube lo que ya es capaz de usar. Un kana o un kanji practicado mueve solo esa pieza. Un vocabulario mueve esa palabra y los kanji que trae, del mismo modo que una frase reparte el puntaje entre las piezas que usa. En la producción libre, que no parte de un ítem, decide una cadena de filtros: la interiorización queda en las piezas usadas bien en ese contexto, y el reconocimiento en las que tienen la forma correcta fuera de él. Para apartar lo ya interiorizado, la regla lee el avance, pero no lo escribe.

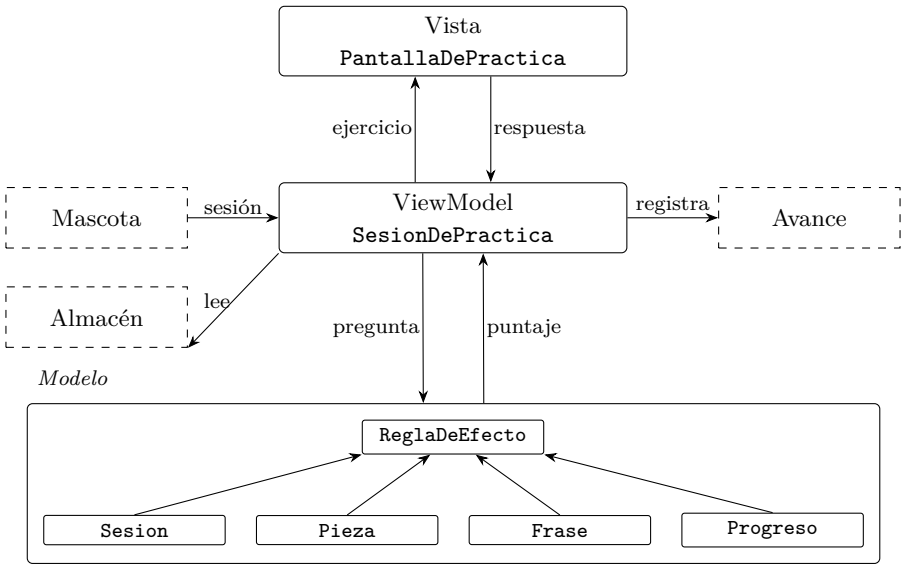


Figura 2. MVVM de Práctica. Recuadro: clases del modelo. Trazo punteado: queda fuera del componente.

4. Fase 4. SOLID

El diseño de Práctica separa el flujo de la sesión, los ejercicios y la regla que reparte los puntajes. La figura 3 muestra esas clases. Cada principio nombra el diseño anterior, en el que esas razones de cambio estaban mezcladas.

Responsabilidad única. `SesionDePractica` cambia si cambia el flujo: abre la sesión, recorre el plan, pide la vista de cada ítem y registra el puntaje cuando la respuesta vuelve. `Sesion` cambia si cambia lo que el aprendiz puede configurar. `ReglaDeEfecto` cambia si cambia qué puntaje sube. Para decidirlo lee `Sesion`, `Pieza` y `Frase`, y consulta el `Progreso`; por eso las flechas salen de la regla hacia esas clases. Devuelve las decisiones y no las guarda. Antes las dos razones vivían en la sesión, y agregar la producción libre obligaba a modificar también el cálculo de los puntajes. `Pieza` cambia si cambia el contenido, y `Progreso` si el aprendiz practica. Antes los dos vivían en la pieza, y actualizar un contenido ponía en riesgo el avance.

Abierto/cerrado. `SesionDePractica` depende de `PantallaDePractica`, que `IIdentificacion`, `IRecuperacion` e `IProduccion` extienden, y de `IEvaluacion`. Esa dependencia es la línea punteada que baja desde la sesión hasta cada interfaz. La sesión solo llama `recoger()`, y la vista de cada ítem sale de un catálogo que asocia el tipo de pieza y el grado con su fábrica. Un ejercicio nuevo es una clase y una entrada en el catálogo, sin modificar la sesión; un

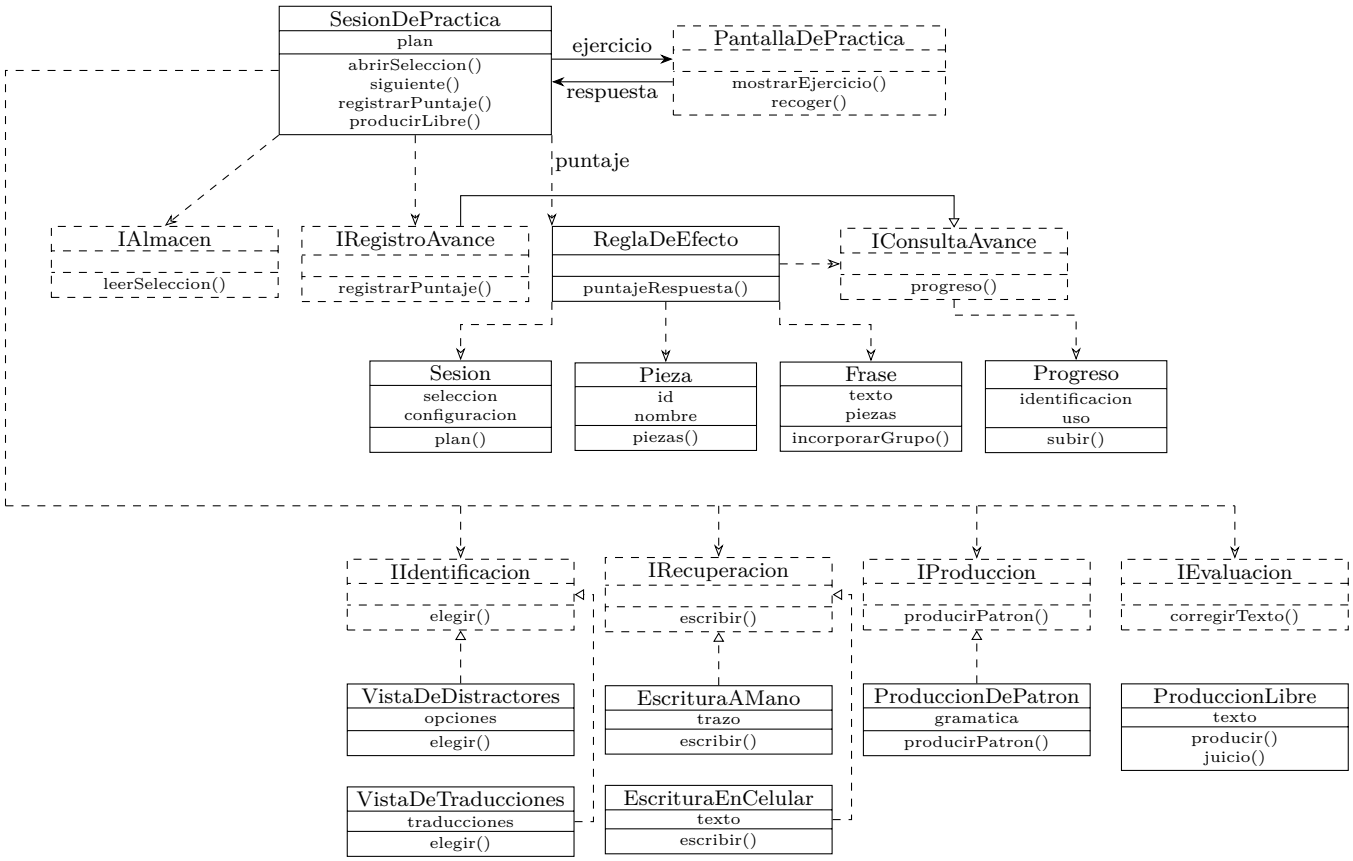


Figura 3. Clases de Práctica para SOLID. Flecha punteada: dependencia. Triángulo punteado: realización. Triángulo continuo: una interfaz extiende a otra.

orden nuevo para el plan es otra clase de `OrdenDeItems`. Antes un condicional dentro de la sesión elegía el ejercicio.

Sustitución de Liskov. El triángulo punteado indica realización. `VistaDeDistractores` y `VistaDeTraducciones` se sustituyen en `IIdentificacion`, porque las dos recogen una elección. `EscrituraAMano` y `EscrituraEnCelular` se sustituyen en `IRecuperacion`. Ubicar partículas y ordenar la frase realizarían también `IIdentificacion`: la sesión solo recibe la elección, así que no distingue cómo se presentó. `ProduccionDePatron` realiza `IProduccion`. Las tres interfaces se sustituyen en `PantallaDePractica`, porque la sesión solo pide mostrar el ejercicio y recoger la respuesta. `ProduccionLibre` no realiza `IEvaluacion` ni `PantallaDePractica`, porque puede subir varias piezas y deja incorporar la frase al grupo. El efecto de un vocabulario tampoco sustituye al de un kanji practicado solo, porque la palabra sube también el kanji que trae. `Kana`, `Kanji`, `Vocabulario` y `Gramatica` sí se sustituyen en `Pieza`: ninguna cambia cómo se recorre la selección ni cómo se registra su avance.

Segregación de interfaces. `IIdentificacion` expone elegir en un conjunto, `IRecuperacion` expone escribir de memoria, `IProduccion` expone producir con un patrón e `IEvaluacion` expone la corrección del agente. Esa última la usa la sesión cuando hay conexión. Sin conexión, la producción libre se queda en su propio juicio y no implementa esa interfaz. Antes una sola interfaz juntaba elegir, escribir y evaluar, y el ejercicio de una pieza implementaba la evaluación sin usarla. Del avance, `IConsultaAvance` expone solo la lectura, y `IRegistroAvance` la extiende con el registro. La regla y los filtros reciben la primera, así que pueden consultar el progreso pero no escribirlo.

Inversión de dependencias. `SesionDePractica` recibe `IAlmacen` e `IRegistroAvance`. Las flechas punteadas salen de la sesión hacia esas interfaces, de modo que no construye el almacén local ni el componente Avance. La regla, a su vez, depende de `IConsultaAvance` y no de la clase que guarda el progreso. Antes el ViewModel creaba esas clases concretas y la sesión no se podía probar sin ellas.

5. Fase 5. Patrones de diseño

Los seis patrones resuelven situaciones que Práctica ya tiene. Hay que abrir el ejercicio sin construirlo dentro de la sesión, practicar una frase sin alterar la que está guardada, recorrer igual un grupo, varios grupos o piezas sueltas, traducir la corrección del agente, repartir los puntajes y filtrar la producción libre antes de ese reparto. En cada ficha se nombra la alternativa que se dejó de lado.

5.1. Factory Method

Problema. La sesión abre un ejercicio según el tipo de pieza y según se trate de identificar o de interiorizar: distractores, traducciones, escritura a mano, escritura en el celular, producción con un patrón o producción libre. Si `SesionDePractica` hace el `new`, cada vista nueva vuelve a modificarla.

Solución. `FabricaDeEjercicio` declara la creación y la subclase devuelve la vista concreta. Un catálogo asocia cada par de tipo de pieza y grado con su fábrica, y la fábrica recibe el ítem que el plan va a practicar. Cada interfaz de la figura 3 tiene su propia jerarquía, porque no

comparten un mismo producto. Se descartó Abstract Factory: no hay una familia de objetos que deba nacer junta, solo la vista del ejercicio que el aprendiz abrió.

Participantes. Creador: `FabricaDeEjercicio`. Creador concreto: `FabricaDeDistractores`. Producto: `IIdentificacion`. Producto concreto: `VistaDeDistractores`. Las traducciones tienen su fábrica concreta y devuelven la misma interfaz; ubicar partículas y ordenar la frase entrarían igual. La recuperación y la producción de un patrón siguen el mismo esquema. La producción libre tiene su propia fábrica, porque no parte de un ítem, y no devuelve `IEvaluacion`.

SOLID. Abierto/cerrado e inversión de dependencias. La sesión recibe la fábrica y no conoce la clase concreta.

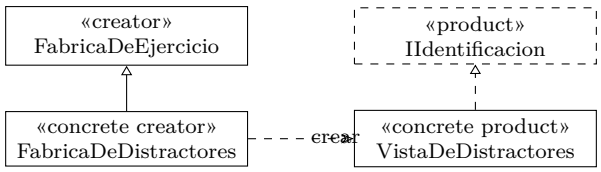


Figura 4. Factory Method al abrir los distractores. Las demás vistas siguen el mismo esquema, cada una con su interfaz.

5.2. Prototype

Problema. Al ordenar una frase, al producir con un patrón o al decidir si una frase nueva entra al grupo, el ejercicio no puede escribir sobre la que está guardada. Si lo hace, el grupo cambia antes de que el aprendiz la incorpore.

Solución. `Frase` se clona y el ejercicio trabaja sobre la copia. Solo `incorporarGrupo()` escribe en el grupo. Se descartó Builder: la frase no se arma con datos opcionales; ya está dicha, y el ejercicio parte de ese enunciado.

Participantes. Prototipo: `Frase`. Cliente: `SesionDePractica`, que pide la copia. El original permanece en el almacén. Como las piezas no guardan el avance ni cambian, la copia comparte las mismas piezas sin riesgo: basta copiar la lista.

SOLID. Responsabilidad única. Guardar la frase y practicar sobre la copia no comparten razón de cambio.

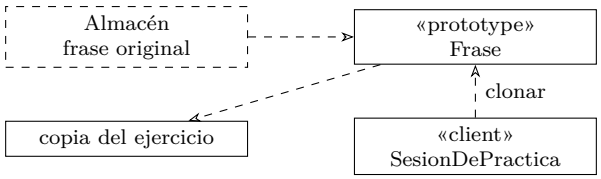


Figura 5. Prototype. El ejercicio trabaja sobre la copia.

5.3. Composite

Problema. La práctica arranca de un grupo, de varios grupos juntados o de piezas sueltas, y necesita sus piezas y, para la gramática, sus frases. Si la sesión distingue esos tres casos, cada forma nueva de elegir qué estudiar la modifica.

Solución. `Grupo` y `Pieza` responden igual a qué piezas y qué frases entran en la práctica. Juntar grupos es un grupo que contiene grupos, y las piezas sueltas son hojas sin frases. Un grupo junta lo de sus hijos sin repetir, de modo que una pieza presente en dos grupos cuenta una sola vez en la sesión. Se descartó Decorator: no se añade comportamiento a una pieza, se recorre una selección como si fuera una sola.

Participantes. Componente: `Seleccion`. Compuesto: `Grupo`. Hoja: `Pieza`. Cliente: `SesionDePractica`.

SOLID. Abierto/cerrado. Sustitución de Liskov: un grupo de grupos ocupa el lugar de un grupo de piezas.

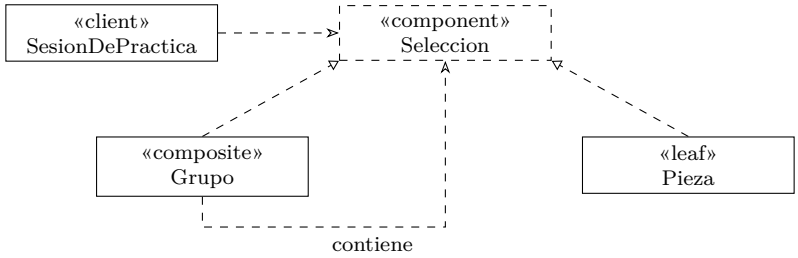


Figura 6. Composite de la selección. Un grupo puede contener otros grupos.

5.4. Adapter

Problema. Con conexión, el proveedor devuelve la corrección en su propio formato. La sesión necesita un resultado que la regla pueda leer: qué piezas aparecen, si la forma es correcta y si el uso cabe en el contexto. El avance del aprendiz no sale del teléfono.

Solución. `AdaptadorDelAgente` realiza `IEvaluacion` y envuelve esa respuesta. El contraste con el avance se hace después, en el teléfono. La pieza no lleva el progreso, así que el adaptador ni siquiera lo tiene a mano. Sin conexión, `ProduccionLibre` no usa esa interfaz y se queda en el juicio del aprendiz. Se descartó Facade: no hay varias clases de un subsistema que ocultar, sino un resultado cuyo formato no coincide con el de la regla.

Participantes. Destino: `IEvaluacion`. Adaptador: `AdaptadorDelAgente`. Adaptado: la respuesta del proveedor. Cliente: `SesionDePractica`.

SOLID. Inversión de dependencias y segregación de interfaces. Evaluar queda fuera de las vistas, y la producción libre no realiza `IEvaluacion`.

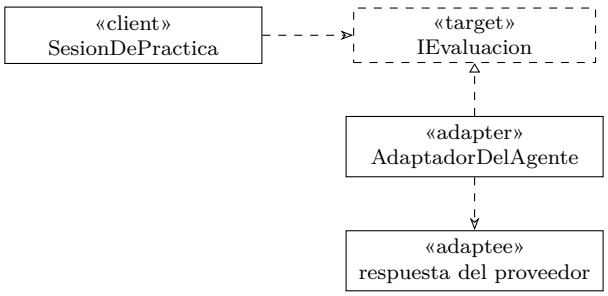


Figura 7. Adapter de la respuesta del proveedor hacia `IEvaluacion`.

5.5. Strategy

Problema. El reparto no es el mismo para todas las piezas. Un kana o un kanji practicado mueve solo esa pieza. Un vocabulario mueve la palabra y el kanji que trae. Una frase de gramática reparte el puntaje entre las piezas que usa. El plan ya dice si cada ítem es de identificación o de uso, así que un condicional dentro de `ReglaDeEfecto` cambiaría con cada reparto nuevo.

Solución. `ReglaDeEfecto` delega en `PoliticaDePuntaje`. La política se elige por el tipo de pieza del ítem, y el mapa de políticas se arma al construir la regla. Cada política devuelve las decisiones —qué pieza sube y qué puntaje—, y la sesión las entrega a Avance. El orden del plan sigue el mismo patrón: `OrdenDeItems` tiene `ComoEnLaSeleccion`, `IdentificarAntesDeUsar`, `MasBajosPrimero`, `MasAltosPrimero` y `AlAzar`, y el aprendiz lo elige al configurar la sesión. Se descartó State: no cambia porque la sesión avance de etapa.

Participantes. Contexto: `ReglaDeEfecto`. Estrategia: `PoliticaDePuntaje`. Estrategias concretas: `EfectoDeKana`, `EfectoDeKanji`, `EfectoDeVocabulario` y `EfectoDeFrase`.

SOLID. Abierto/cerrado y responsabilidad única. Un reparto nuevo es una clase nueva y no reabre la sesión.

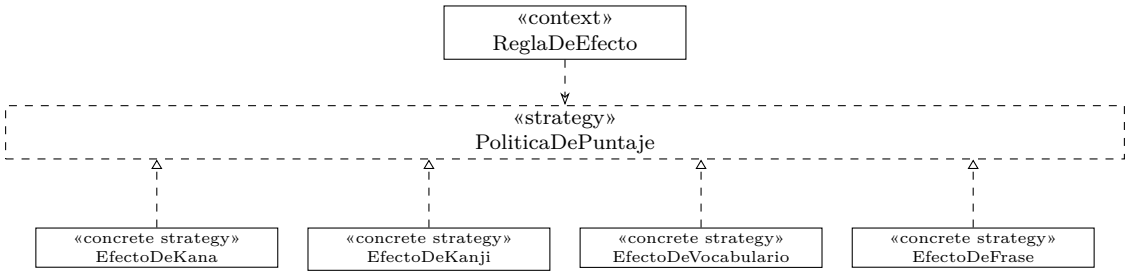


Figura 8. Strategy del reparto. El plan ya fijó el grado de cada ítem.

5.6. Chain of Responsibility

Problema. En la producción libre no hay un ítem: el texto pone en juego varias piezas, y cada una puede subir de forma distinta. Antes de repartir, la producción libre aparta el hiragana y el katakana ya interiorizados, según el progreso que consulta en `IConsultaAvance`, y separa la forma correcta del uso en el contexto pedido. Eso vale tanto si el resultado viene del agente como si viene del juicio del aprendiz. Si esos filtros viven en una política, un filtro nuevo la modifica.

Solución. Cada filtro decide sobre la pieza o la pasa al siguiente. Se descartó Template Method: la cantidad y el orden de los filtros cambian, y no son pasos fijos de una misma receta.

Participantes. Manejador: `FiltroDePuntaje`. Manejadores concretos: `FiltroDeKanaDominado`, `FiltroDeForma` y `FiltroDeContexto`. Cliente: `ReglaDeEfecto`, que arma la cadena y toma como reparto lo que la cadena decide.

SOLID. Abierto/cerrado y responsabilidad única. Cada filtro tiene una sola decisión.

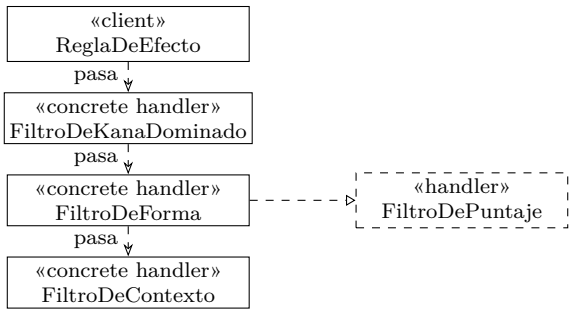


Figura 9. Filtros de la producción libre. Los tres realizan `FiltroDePuntaje`.

6. Fase 6. Código

Práctica está escrita en Kotlin, que es el lenguaje del celular. Los paquetes `vista`, `viewmodel` y `modelo` repiten el MVVM de la fase 3. El almacén, el registro de avance y el proveedor quedan en `infraestructura`, en memoria: cada vista recibe la respuesta que el aprendiz ya dio.

`SesionDePractica` arma el plan con la configuración de la sesión, abre la vista de cada ítem con el catálogo de fábricas, trabaja la frase de la gramática sobre una copia, le pide las decisiones a `ReglaDeEfecto` y las entrega a `IRegistroAvance`, que guarda el progreso por el identificador de la pieza. Con conexión, `AdaptadorDelAgente` traduce la respuesta del proveedor y la solicitud solo lleva el texto. Sin conexión, la producción libre usa su juicio y la misma cadena de filtros. Ubicar partículas y ordenar la frase quedan descritos en el diseño, pero todavía no tienen vista en el código. Las pruebas están en `PracticaTest.kt` y corren con `./gradlew test`. Una de ellas define `EfectoDeLectura` y la registra para el kanji al construir la regla: la regla devuelve sus decisiones, y la clase `ReglaDeEfecto` no se modifica.

Patrón	Archivo	Clases
Factory Method	`FabricaDeEjercicio.kt`, `CatalogoDeEjercicios.kt`	`FabricaDeEjercicio`, `FabricaDeDistractores`, `CatalogoDeEjercicios`
Prototype	`Frase.kt`	`Frase`
Composite	`Seleccion.kt`, `Pieza.kt`	`Seleccion`, `Grupo`, `Pieza`
Adapter	`AdaptadorDelAgente.kt`	`AdaptadorDelAgente`, `IEvaluacion`
Strategy	`PoliticaDePuntaje.kt`, `OrdenDeItems.kt`	`ReglaDeEfecto`, `PoliticaDePuntaje`, `EfectoDeKana`, `EfectoDeKanji`, `EfectoDeVocabulario`, `EfectoDeFrase`, `OrdenDeItems`
Chain of Responsibility	`FiltroDePuntaje.kt`	`FiltroDePuntaje`, `FiltroDeKanaDominado`, `FiltroDeForma`, `FiltroDeContexto`

7. Fase 7. Prueba de evolución

Cuatro cambios que el dominio puede pedir. En cada uno se dice qué clase aparece, qué se modifica y qué sigue igual.

Otro proveedor de IA. La mascota pasa a corregir con un proveedor cuyo formato no es el de ahora. Aparece un adaptador nuevo que realiza `IEvaluacion` y traduce esa respuesta al resultado que la regla ya lee. Al armar la sesión se le entrega ese adaptador. `SesionDePractica`, `ProduccionLibre`, `ReglaDeEfecto` y la cadena de filtros no se tocan. Sin conexión, el juicio del aprendiz sigue igual, porque ese camino no pasa por el proveedor.

Un compuesto. Una palabra compuesta debe subir su puntaje, el de las palabras que la forman y el kanji de cada una. Aparece `EfectoDeCompuesto`, otra `PoliticaDePuntaje`. Se registra para el vocabulario al construir la regla, con el mapa extra de `porDefecto`, y deja las palabras simples a `EfectoDeVocabulario`. `ReglaDeEfecto`, `EfectoDeVocabulario`, `EfectoDeKanji`, `EfectoDeKana` y `EfectoDeFrase` no se modifican. La sesión tampoco: sigue pidiendo el puntaje y no elige el reparto.

El kanji ya interiorizado. En la producción libre, un kanji que el aprendiz ya usa bien sale del foco, igual que el hiragana y el katakana. Aparece `FiltroDeKanjiDominado`. Los filtros que ya existen no cambian: el nuevo decide o pasa la pieza. Quien arma la cadena lo inserta antes de `FiltroDeForma`. Si esa cadena entra por el constructor de `ReglaDeEfecto`, la clase no se modifica. `porDefecto`, que hoy deja armados los tres filtros, es el único método que pasa a nombrar el cuarto. Fuera de la producción libre no hace falta: el umbral de la sesión ya deja fuera lo que el aprendiz domina.

Repaso espaciado. Para que las frases no se olviden, como pide RF2, cada grado de una pieza pasa a tener su propio calendario: cuándo se repasó, cada cuánto y cuándo vuelve. Aparece `AlgoritmoDeRepaso`, una estrategia que `Progreso` usa para calcular el próximo repaso; cambiar de algoritmo no toca nada más. Aparece `PendientesPrimero`, otro `OrdenDeItems` que pone primero lo que toca repasar hoy, y el aprendiz lo elige al configurar la sesión como los demás órdenes. El tiempo entra por una interfaz `Reloj`, para probar sin esperar días. Lo que sí se modifica es `Decision`: además de la pieza y el grado, lleva si la respuesta fue un acierto o un fallo, porque un fallo adelanta el repaso. `SesionDePractica` pasa a registrar también las respuestas fallidas, que hoy descarta, y los filtros distinguen un fallo de una pieza que no aplica. `ReglaDeEfecto` y las políticas siguen decidiendo qué pieza y qué grado se mueven. Queda abierto si practicar una frase cuenta como repaso completo de cada pieza o solo parcial.

8. Trazabilidad

Cada requisito de la fase 1 queda unido al componente que lo atiende, a las clases, al patrón y al principio. Donde el requisito vive fuera de Práctica, no se le asigna un patrón.

Requisito	Componente	Clases	Patrón	SOLID
RF1. Ruta sugerida	Grupos	`Grupo`, `Seleccion`	Composite	Responsabilidad única
RF2. Incorporar frases	Grupos, Práctica	`Frase`, `Grupo`	Prototype	Responsabilidad única
RF3. Dos puntajes por pieza	Práctica, Avance	`Pieza`, `Progreso`, `ReglaDeEfecto`, `IRegistroAvance`	Strategy	Responsabilidad única, inversión
RF4. Practicar una pieza	Práctica	`SesionDePractica`, `Sesion`, `CatalogoDeEjercicios`, `FabricaDeEjercicio`, `PantallaDePractica`	Factory Method, Strategy	Abierto/cerrado, Liskov
RF5. Practicar una frase	Práctica	`Frase`, `EfectoDeFrase`	Strategy, Prototype	Abierto/cerrado
RF6. Ver el avance	Avance	`IRegistroAvance`, `IConsultaAvance`	—	Inversión, segregación
RF7. Lecturas	Lecturas, Detalle	—	—	Responsabilidad única
RF8. Estudiar sin conexión	Almacén, Práctica	`IAlmacen`, `ProduccionLibre`	—	Segregación, inversión
RF9. Ejercicios de la mascota	Mascota, Práctica	`AdaptadorDelAgente`, `IEvaluacion`, `FiltroDePuntaje`	Adapter, cadena	Inversión, segregación, abierto/cerrado
RF10. Anuncios o pago	Módulo mascota	—	—	Responsabilidad única

9. Preguntas de reflexión

1. ¿Qué patrón resultó más natural y cuál fue el más difícil de justificar? El más natural fue Strategy. Los repartos ya estaban en el dominio: el kana y el kanji solos, el vocabulario con su kanji y la frase. La regla delega en la política que corresponde al tipo de pieza del ítem, y el orden del plan se elige del mismo modo. El más difícil fue Prototype. La copia existe para que ordenar una frase, o decidir si entra al grupo, deje intacta la que está guardada, y solo `incorporarGrupo` escribe en el grupo. El objeto es pequeño, y como las piezas ya no guardan avance la copia de la lista basta, así que la distancia con una lista temporal es corta. Se mantuvo porque el original sigue en el almacén aunque el ejercicio lo reordene.

2. ¿En qué se parecen Strategy y State? Los dos esconden un comportamiento detrás de una interfaz, y el contexto deja de llenarse de condicionales. En Strategy la política sale del tipo de pieza y no cambia: `EfectoDeKana`, `EfectoDeKanji`, `EfectoDeVocabulario` o `EfectoDeFrase`. State correspondería a un objeto que cambia de comportamiento porque pasó de etapa, por ejemplo una pieza que pasa de identificar a interiorizar. En Práctica esa transición no ocurre: una sesión puede tener ítems de identificar y de usar, pero cada ítem trae su grado desde el plan, y el reparto no se reemplaza porque la sesión avance.

3. ¿En qué escala trabajan el estilo orientado a eventos y Observer? Observer trabaja dentro de un mismo proceso: un objeto avisa a otros que comparten memoria. El estilo orientado a eventos trabaja entre partes desplegadas por separado, y el aviso viaja por un canal común. En Musubi Japan el celular es un monolito modular y el único servicio aparte es la mascota. No hay un canal de eventos entre componentes. Si la pantalla de avance escuchara el puntaje de la sesión, esa sería la escala de Observer. Partir grupos, práctica y avance en servicios

sería la escala de los eventos, y la fase 2 la dejó de lado porque esas funciones tienen que responder sin red.

4. ¿Qué patrón se eliminaría en un sistema mucho más pequeño, y qué costo tiene uno innecesario? Con un solo ejercicio y un solo grupo, el primero en salir sería Factory Method. La sesión podría construir esa única vista. Dejar el patrón sin una variedad real obliga a cruzar una fábrica antes de ver el ejercicio, y aparece una clase más por cada vista. Aquí se queda porque identificar, recuperar, producir con un patrón y producir libre ya son vistas distintas, y una nueva entra al catálogo sin modificar `SesionDePractica`.

5. ¿Qué partes se mantendrían al pasar el monolito a microservicios, o al revés? Se mantiene el borde que ya está aislado. `IEvaluacion` y el adaptador siguen traduciendo al proveedor, y la sesión no conoce el formato. La regla, las políticas y la cadena siguen juntas dentro de Práctica, porque el puntaje se calcula en el teléfono y el avance no sale en la solicitud. `IAlmacen` e `IRegistroAvance` permiten cambiar la implementación sin tocar la sesión. En ese traslado seguirían en el celular, en memoria, para responder sin red. El camino de vuelta, reunir un servicio remoto de práctica con el teléfono, usa otra vez esas interfaces con las clases de `infraestructura` que ya existen.